

Quiz 2 Study Guide — Lectures 4–7

CSCE 775 — Spring 2026

J.C. Vaught

Quiz format: 1 math problem (50%) + conceptual questions (50%). Strategy: nail the conceptual questions, get partial credit on the math.

Part I

Lecture 4: Model-Free RL

Covers Sutton & Barto Ch. 5.1–5.7, 6.1–6.5, 7.1–7.3

1 Big Picture

Model-free RL means learning from *experience* without knowing the transition function $p(s', r \mid s, a)$. Two tasks:

- **Prediction (policy evaluation):** estimate v_π or q_π for a given π
- **Control:** find the optimal policy π_*

Three families of methods, each with a different *target* for the update:

Method	Target	Bias	Variance
Monte Carlo	G_t (full return)	None	High
TD(0)	$R_{t+1} + \gamma V(S_{t+1})$	Yes (bootstrap)	Low
n -step TD	$R_{t+1} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$	Medium	Medium

2 Monte Carlo Methods (Ch. 5)

2.1 MC Prediction

Estimate $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ by averaging *sampled returns*.

MC Incremental Update: $V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$

- **First-visit MC:** average G_t only the *first* time s appears in an episode.
- **Every-visit MC:** average G_t every time s appears.
- Must wait until the episode ends to compute G_t . Does *not* work for continuing (non-episodic) tasks.

2.2 MC Control (On-Policy)

Use $Q(s, a)$ instead of $V(s)$ because we cannot derive a policy from V without a model.

Policy from Q: $\pi(s) = \arg \max_a Q(s, a)$

ε -greedy exploration:

$$\pi(a | s) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(s)| & \text{if } a = \arg \max_{a'} Q(s, a') \\ \varepsilon/|\mathcal{A}(s)| & \text{otherwise} \end{cases}$$

The on-policy MC control algorithm: generate episode $\rightarrow$ compute returns backward $\rightarrow$ update $Q(S_t, A_t)$ $\rightarrow$ improve policy to ε -greedy w.r.t. new Q .

2.3 On-Policy vs Off-Policy

- **Behavior policy b :** the policy used to collect data.
- **Target policy π :** the policy we want to evaluate/improve.
- **On-policy:** $b = \pi$.
- **Off-policy:** $b \neq \pi$. Can reuse data from other policies. Requires importance sampling.

3 Temporal-Difference Learning (Ch. 6)

3.1 TD(0) Prediction

Key idea: update toward a *bootstrapped* target instead of waiting for G_t .

TD(0) Update: $V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{TD target}}$

The quantity $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is the **TD error**.

MC vs TD:

- MC: unbiased, high variance, needs full episodes.
- TD: biased (initially), low variance, learns online step-by-step.
- TD bootstraps from its own estimate $\Rightarrow$ can propagate value information faster.

3.2 SARSA — On-Policy TD Control

SARSA Update: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$

Name comes from the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.

Both behavior and target policy are ε -greedy. Choose A_{t+1} from S_{t+1} using ε -greedy *before* the update.

3.3 Q-Learning — Off-Policy TD Control

$$\text{Q-Learning Update: } Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Key difference: uses $\max_a Q(S_{t+1}, a)$ (greedy) instead of $Q(S_{t+1}, A_{t+1})$ (ε -greedy).

- Behavior policy: ε -greedy (explores).
- Target policy: greedy (deterministic).
- Off-policy because behavior $\neq$ target.

Cliff Walking Example:

- SARSA learns a *safer* path (avoids cliff) because it accounts for its own ε -greedy exploration.
- Q-learning learns the *optimal* path (close to cliff) because the target policy is greedy.

4 n -Step Methods (Ch. 7)

$$n\text{-step return: } G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- $n = 1$: TD(0).
- $n = \infty$: Monte Carlo.
- Typical sweet spot: $n \approx 5$.

TD(λ) averages over *all* n -step returns with exponentially decaying weights $(1 - \lambda)\lambda^{n-1}$. When $\lambda = 0$ you get TD(0); when $\lambda = 1$ you get MC.

5 Unified View: DP vs TD

	Full Backup (DP)	Sample Backup (TD)
Bellman Expectation for v_π	Policy Evaluation	TD Learning
Bellman Expectation for q_π	Q-Policy Iteration	SARSA
Bellman Optimality for q_*	Q-Value Iteration	Q-Learning

Key difference: DP uses the full model (all transitions); TD samples one transition.

Part II

Lecture 5: Deep Learning Fundamentals

6 Linear Models

6.1 Linear Regression

$$f(\mathbf{x}, \mathbf{w}, b) = \mathbf{w}^\top \mathbf{x} + b \quad \text{Loss: } \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (y_i - f(\mathbf{x}_i, \boldsymbol{\theta}))^2$$

Analytical solution: $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

Gradient descent update: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$

6.2 Logistic Regression (Binary Classification)

$$\text{Sigmoid: } \sigma(z) = \frac{1}{1+e^{-z}} \quad \text{Derivative: } \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$. Decision boundary where $\mathbf{w}^\top \mathbf{x} = 0$.

Loss: negative log-likelihood (cross-entropy):

$$\mathcal{L} = - \sum_i [y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i))]$$

6.3 Softmax (Multi-class)

$$P(y = i \mid \mathbf{x}) = \frac{e^{\mathbf{w}_i^\top \mathbf{x}}}{\sum_{j=1}^C e^{\mathbf{w}_j^\top \mathbf{x}}}$$

7 Neural Networks

7.1 Architecture

A 2-layer network: $f(\mathbf{x}) = \mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x})$

Activation functions must be non-linear, otherwise the network collapses to a linear model regardless of depth.

7.2 Common Activation Functions

Name	Formula	Notes
Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$	Output in $(0, 1)$; vanishing gradient
Tanh	$\tanh(x)$	Output in $(-1, 1)$
ReLU	$\max(0, x)$	Most common; dead neurons possible
Leaky ReLU	$\max(0.1x, x)$	Fixes dead neuron problem

7.3 Universal Function Approximation

With enough hidden units, a neural network can approximate *any* continuous function to arbitrary precision. Does *not* guarantee convergence of training.

7.4 Backpropagation

Chain rule applied layer by layer. For output layer weights:

$$\frac{\partial E}{\partial W_{ij}^{(2)}} = -(y_i - \hat{y}_i) h_j$$

For hidden layer weights, errors propagate backward through the chain rule.

7.5 Hyperparameters vs Parameters

- **Parameters:** learned from data (weights, biases).
- **Hyperparameters:** set before training (learning rate, # layers, # neurons, ε , etc.).

8 Overfitting and Regularization

Technique	How it works
Train/Val/Test split	Monitor val loss; never train on test
Weight regularization	Add $\lambda \sum W_{ij}^2$ to loss
Dropout	Randomly zero out neurons during training
Data augmentation	Rotations, flips, noise to increase data
Early stopping	Stop when validation error starts rising

9 Optimization

Loss surface is non-convex: local minima, saddle points.

Optimizer	Update Rule
SGD	$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla E$
SGD + Momentum	$\mathbf{v} \leftarrow \mu \mathbf{v} + \alpha \nabla E; \quad \mathbf{w} \leftarrow \mathbf{w} - \mathbf{v}$
ADAM	Adaptive lr per parameter using mean & variance of gradients

Stochastic GD: compute gradients on random mini-batches instead of full dataset.

9.1 Residual Networks (ResNets)

Skip connections: $\text{output} = F(\mathbf{x}) + \mathbf{x}$. Smooths the loss surface; enables training very deep networks (100+ layers).

9.2 Batch Normalization

Normalize activations to mean 0, std 1 within each mini-batch. Stabilizes training, allows larger learning rates.

Part III

Lecture 6: PyTorch

10 Key Concepts (Likely Conceptual Questions)

1. **Automatic differentiation:** define forward pass; PyTorch computes backward pass (gradients) automatically via computation graph.
2. **Tensor:** multi-dimensional array (like numpy ndarray), but tracks gradients.
3. **Computation graph:** directed graph of operations. Forward pass builds it; `loss.backward()` traverses it backward.
4. `.detach()`: removes a tensor from the computation graph (stops gradient flow).
5. **Data type:** PyTorch expects `float32`. Convert before or after creating tensors.
6. `loss.item()`: extract scalar value without keeping the computation graph (prevents memory leak).
7. `nnet.train()` vs `nnet.eval()`: training mode enables dropout and batch norm updates; eval mode disables them.
8. **Saving/loading:** `torch.save(nnet.state_dict(), path)` saves only parameters, not architecture.

11 Training Loop Pattern

1. `optimizer.zero_grad()` — clear old gradients
2. Forward pass: `output = nnet(input)`
3. Compute loss: `loss = criterion(output, target)`
4. Backward pass: `loss.backward()`
5. Update weights: `optimizer.step()`

12 ResNet in PyTorch

Residual block: two linear layers + skip connection. Requires input dim = hidden dim (or use a linear projection). Forward: $\mathbf{x} \leftarrow \text{ReLU}(F(\mathbf{x}) + \mathbf{x})$.

13 Gradient Checking

Finite differences: $\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta_i + h) - f(\theta_i)}{h}$ for small h (e.g., 10^{-5}). Verify each layer independently, then compose.

Part IV

Lecture 7: DAVI and Deep Q-Networks

14 Deep Approximate Value Iteration (DAVI)

When the state space is too large for a table, approximate $v_*(s)$ with a neural network $v_\theta(s)$.

$$\text{DAVI Target: } y = \begin{cases} \max_a (r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_\theta(s')) & \text{if } s \text{ non-terminal} \\ 0 & \text{if } s \text{ terminal} \end{cases}$$

$$\text{Loss: } L(\theta) = \frac{1}{2}(y - v_\theta(s))^2 \quad \text{Gradient: } \nabla_\theta L = (y - v_\theta(s)) \nabla_\theta v_\theta(s)$$

Do NOT differentiate y with respect to θ even though y depends on θ . Treat y as a fixed target.

14.1 Target Network

Problem: target y changes every iteration (non-stationary).

Solution: maintain a separate **target network** v_{θ^-} whose parameters are periodically copied from θ . Use θ^- to compute y , train θ .

15 Deep Q-Networks (DQN)

Approximate $q_*(s, a)$ with a neural network. Input: state s . Output: Q -values for all actions (one forward pass).

$$\text{DQN Target: } y = \begin{cases} r + \gamma \max_{a'} q_{\theta^-}(s', a') & \text{non-terminal} \\ r & \text{terminal} \end{cases}$$

$$\text{DQN Loss: } L(\theta) = \frac{1}{2}(y - q_\theta(s, a))^2$$

15.1 Three Key DQN Innovations

1. **Target network** (θ^- updated periodically) — stabilizes training by making targets less non-stationary.
2. **Experience replay buffer** — store (s, a, r, s') tuples; sample random mini-batches to decorrelate training data and prevent catastrophic forgetting.
3. **Huber loss** — clips gradients for large TD errors, preventing instability from large rewards.

15.2 DQN Ablation (know this!)

Both replay buffer AND target network are essential. Removing either one drastically hurts performance. DQN also vastly outperforms linear function approximators.

16 DQN Improvements

Method	Key Idea
Double Q-Learning	Two Q-networks: one selects the action, the other evaluates it. Fixes <i>maximization bias</i> . $y = r + \gamma q_{\theta_2}(s', \arg \max_{a'} q_{\theta_1}(s', a'))$
Dueling DQN	Split network into value stream $v_{\theta}(s)$ and advantage stream $A_{\theta}(s, a)$. $q(s, a) = v_{\theta}(s) + A_{\theta}(s, a)$. Advantage: $A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s)$.
Prioritized Replay	Sample transitions with large TD error more frequently.
Rainbow	Combines DQN + Double + Dueling + Prioritized + others. Best overall.
Agent57	First to beat humans on all 57 Atari games. Uses intrinsic curiosity + adaptive discount.

Part V

Likely Derivation Problems

The math problem is 50% and is a pure symbolic derivation (no numbers). Here are the most likely candidates with full step-by-step derivations.

17 Derivation 1: MC Incremental Update from Running Average

“Show that the running-average MC update can be written in incremental form.”

Start from the sample-mean definition after n visits to state s :

$$V_n = \frac{1}{n} \sum_{k=1}^n G_k$$

Split off the n -th return:

$$V_n = \frac{1}{n} \left(G_n + \sum_{k=1}^{n-1} G_k \right) = \frac{1}{n} \left(G_n + (n-1) V_{n-1} \right) \quad (1)$$

$$= \frac{1}{n} G_n + \frac{n-1}{n} V_{n-1} = \frac{1}{n} G_n + V_{n-1} - \frac{1}{n} V_{n-1} \quad (2)$$

$$= V_{n-1} + \frac{1}{n} (G_n - V_{n-1}) \quad (3)$$

$$V_n = V_{n-1} + \alpha_n (G_n - V_{n-1}) \quad \text{where } \alpha_n = 1/n$$

Generalizing to a constant step size: $V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$

18 Derivation 2: ε -Greedy Policy Improvement

“Prove that for any ε -greedy policy π , the ε -greedy policy π' w.r.t. q_π is an improvement: $v_{\pi'}(s) \geq v_\pi(s)$.”

Start with the expected value under π' :

$$q_\pi(s, \pi'(s)) = \sum_a \pi'(a | s) q_\pi(s, a) = \frac{\varepsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \max_a q_\pi(s, a) \quad (4)$$

Since $\max_a q_\pi(s, a) \geq \sum_a w_a q_\pi(s, a)$ for any convex weights w_a summing to 1, substitute $w_a = \frac{\pi(a|s) - \varepsilon/|\mathcal{A}|}{1 - \varepsilon}$:

$$\geq \frac{\varepsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \sum_a \frac{\pi(a | s) - \varepsilon/|\mathcal{A}|}{1 - \varepsilon} q_\pi(s, a) \quad (5)$$

$$= \frac{\varepsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) + \sum_a (\pi(a | s) - \varepsilon/|\mathcal{A}|) q_\pi(s, a) \quad (6)$$

$$= \sum_a \pi(a | s) q_\pi(s, a) = v_\pi(s) \quad (7)$$

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \text{ for all } s. \text{ By the } \mathbf{policy \text{ improvement theorem}}, \text{ this implies } v_{\pi'}(s) \geq v_\pi(s) \text{ for all } s. \quad \square$$

19 Derivation 3: Sigmoid Derivative

“Derive $\frac{d}{dz}\sigma(z)$ where $\sigma(z) = \frac{1}{1+e^{-z}}$.”

$$\sigma(z) = (1 + e^{-z})^{-1} \quad (8)$$

$$\frac{d\sigma}{dz} = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} \quad (9)$$

$$= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z) \cdot \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} \quad (10)$$

$$= \sigma(z)(1 - \sigma(z)) \quad (11)$$

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$$

20 Derivation 4: Gradient of Logistic Regression Loss

“Derive the gradient of the cross-entropy loss for logistic regression.”

The log-likelihood for one sample:

$$\ell = y \log \sigma(\mathbf{w}^\top \mathbf{x}) + (1 - y) \log(1 - \sigma(\mathbf{w}^\top \mathbf{x}))$$

Using $\sigma' = \sigma(1 - \sigma)$ and abbreviating $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x})$:

$$\frac{\partial \ell}{\partial \mathbf{w}} = y \cdot \frac{\hat{y}(1 - \hat{y})}{\hat{y}} \mathbf{x} + (1 - y) \cdot \frac{-\hat{y}(1 - \hat{y})}{1 - \hat{y}} \mathbf{x} \quad (12)$$

$$= y(1 - \hat{y}) \mathbf{x} - (1 - y)\hat{y} \mathbf{x} = (y - \hat{y}) \mathbf{x} \quad (13)$$

The loss is *negative* log-likelihood, so the gradient of the loss over N samples:

$$\nabla_{\mathbf{w}} \mathcal{L} = -\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i) \mathbf{x}_i = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \mathbf{x}_i \quad \text{where } \hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

21 Derivation 5: Backpropagation for a 2-Layer Network

“Derive the weight gradients for a 2-layer network.”

Network: $\hat{\mathbf{y}} = \mathbf{W}^{(2)} \mathbf{h}$ where $\mathbf{h} = \sigma(\mathbf{W}^{(1)} \mathbf{x})$. Loss: $E = \frac{1}{2} \|\mathbf{y} - \hat{\mathbf{y}}\|^2 = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2$.

Output layer weights (apply chain rule once):

$$\hat{y}_i = \sum_j W_{ij}^{(2)} h_j \quad (14)$$

$$\frac{\partial E}{\partial W_{ij}^{(2)}} = \frac{\partial E}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial W_{ij}^{(2)}} = -(y_i - \hat{y}_i) \cdot h_j \quad (15)$$

Hidden layer weights (chain rule through two layers):

$$h_j = \sigma\left(\sum_k W_{jk}^{(1)} x_k\right) \quad (16)$$

$$\frac{\partial E}{\partial W_{jk}^{(1)}} = \sum_i \frac{\partial E}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial h_j} \cdot \frac{\partial h_j}{\partial W_{jk}^{(1)}} = \sum_i [-(y_i - \hat{y}_i)] W_{ij}^{(2)} \sigma'\left(\sum_k W_{jk}^{(1)} x_k\right) x_k \quad (17)$$

Pattern: error at output $\times$ weight connecting layers $\times$ activation derivative $\times$ input.
Errors “back-propagate” through each layer via the chain rule.

22 Derivation 6: DAVI / Approximate Value Iteration Gradient

“Derive the gradient of the approximate value iteration loss.”

Target (treated as constant): $y = \max_a (r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_{\theta^-}(s'))$

Loss for a single state:

$$L(\theta) = \frac{1}{2} (y - v_{\theta}(s))^2$$

Apply the chain rule (do *not* differentiate y w.r.t. θ):

$$\nabla_{\theta} L(\theta) = \frac{\partial}{\partial \theta} \frac{1}{2} (y - v_{\theta}(s))^2 = (y - v_{\theta}(s)) \cdot (-\nabla_{\theta} v_{\theta}(s)) \quad (18)$$

$\nabla_{\theta} L(\theta) = -(y - v_{\theta}(s)) \nabla_{\theta} v_{\theta}(s)$
The gradient descent update is then: $\theta \leftarrow \theta - \alpha \nabla_{\theta} L = \theta + \alpha (y - v_{\theta}(s)) \nabla_{\theta} v_{\theta}(s)$
Key: y depends on θ^- (frozen target network), NOT on θ , so it is treated as a constant.

For N states (batched):

$$L(\theta) = \frac{1}{2N} \sum_{i=1}^N (y_i - v_{\theta}(s_i))^2 \quad \Rightarrow \quad \nabla_{\theta} L = -\frac{1}{N} \sum_{i=1}^N (y_i - v_{\theta}(s_i)) \nabla_{\theta} v_{\theta}(s_i)$$

23 Derivation 7: Linear Regression Gradient (Closed Form)

“Derive the analytical solution for linear regression.”

Model: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$. Loss:

$$\mathcal{L}(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 = (\mathbf{X}\mathbf{w} - \mathbf{y})^{\top} (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Expand:

$$\mathcal{L} = \mathbf{w}^{\top} \mathbf{X}^{\top} \mathbf{X} \mathbf{w} - 2\mathbf{w}^{\top} \mathbf{X}^{\top} \mathbf{y} + \mathbf{y}^{\top} \mathbf{y} \quad (19)$$

Take gradient and set to zero:

$$\nabla_{\mathbf{w}} \mathcal{L} = 2\mathbf{X}^{\top} \mathbf{X} \mathbf{w} - 2\mathbf{X}^{\top} \mathbf{y} = \mathbf{0} \quad (20)$$

$$\mathbf{w}^* = (\mathbf{X}^{\top} \mathbf{X})^{-1} \mathbf{X}^{\top} \mathbf{y}$$

Part VI

Rapid-Fire Conceptual Review

1. **Why Q instead of V for model-free control?** Cannot derive π from $V(s)$ without knowing $p(s'|s, a)$. With $Q(s, a)$, just take $\arg \max_a$.
2. **MC vs TD bias/variance?** MC: unbiased, high variance. TD: biased (bootstraps), low variance.
3. **Can MC handle continuing tasks?** No. Must wait for episode to end.
4. **SARSA vs Q-learning?** SARSA is on-policy (uses $Q(S', A')$); Q-learning is off-policy (uses $\max_a Q(S', a)$). In cliff walking, SARSA is safer, Q-learning is optimal.
5. **What is bootstrapping?** Using current estimates of value to update other estimates (TD does this; MC does not).
6. **What is the TD error?** $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$.
7. **Why use ϵ -greedy?** Ensures exploration. Pure greedy can get stuck.
8. **Why must activations be non-linear?** Otherwise, any depth of network collapses to a single linear transformation.
9. **What is ReLU?** $\max(0, x)$. Simple, efficient, avoids vanishing gradient (for $x > 0$).
10. **What is a residual connection?** $\text{output} = F(\mathbf{x}) + \mathbf{x}$. Enables training deep networks by smoothing the loss surface.
11. **What is batch normalization?** Normalize layer inputs to mean 0, std 1 per mini-batch. Stabilizes and accelerates training.
12. **Overfitting solutions?** Regularization, dropout, data augmentation, early stopping, simpler model.
13. **Why experience replay?** Decorrelates training samples. Prevents catastrophic forgetting. Improves data efficiency.
14. **Why target network?** Stabilizes training by keeping the target fixed for many updates, reducing non-stationarity.
15. **What is maximization bias?** Q-learning overestimates Q-values because it uses max for both selection and evaluation. Double Q-learning fixes this.
16. **What is the advantage function?** $A_\pi(s, a) = q_\pi(s, a) - v_\pi(s)$. How much better action a is than average.
17. **Dueling DQN?** Separate streams for $v(s)$ and $A(s, a)$, combined as $q(s, a) = v(s) + A(s, a)$.
18. **What does `.detach()` do in PyTorch?** Removes a tensor from the computation graph, stopping gradient flow.
19. **`nnet.train()` vs `nnet.eval()`?** Train mode enables dropout and batch norm updates. Eval mode disables them for deterministic inference.
20. **Why `loss.item()` not `loss`?** Using `loss` keeps the computation graph in memory, causing a memory leak.
21. **Universal approximation theorem?** A neural network with enough hidden units can approximate any continuous function. Does NOT guarantee we can find the right weights via training.
22. **SGD vs full gradient descent?** SGD uses random mini-batches instead of full dataset. Faster per

step, noisier gradients, but often converges well.

- 23. **What is n -step TD?** Intermediate between TD(0) and MC. Uses n actual rewards then bootstraps. $n = 1$ is TD(0), $n = \infty$ is MC.
- 24. **DQN ablation key finding?** Both replay buffer AND target network are essential. Removing either one drastically hurts performance.
- 25. **Robbins-Monro conditions?** For convergence: $\sum \alpha_n = \infty$ (steps large enough to overcome initial conditions) and $\sum \alpha_n^2 < \infty$ (steps shrink enough to converge).